2026 开源鸿蒙大学生创新大赛 · 赛道一

HapTest

面向 OpenHarmony 应用的多模态智能测试系统

OpenHarmony 应用的智能测试与性能观测基础设施

参赛作品简介

北京航空航天大学 · SMAT 团队 · 刘发荣

2026 年 11 月

作品仓库：github.com/growvv/HapTest-2026

GitCode：gitcode.com/rogn/HapTest-2026

一、作品定位

HapTest 是面向 OpenHarmony 应用的多模态智能测试系统，也是 OpenHarmony 应用的智能测试与性能观测基础设施。多模态观察是它的入口：把控件树、截图、运行日志与任务语义放进同一次观察，向下打通 HDC、uitest agent、Hypium RPC 与 ArkUI Inspector，向上提供页面迁移图（PTG）、可扩展测试策略、haptest harmony 命令行与 JSON-lines 会话，并探索基于 DevEco trace 的逐帧掉帧观测。作品的目标不是再写一个点击脚本，而是把“人的测试意图”变成“可执行、可复现、可追溯的证据”。

二、背景与问题

OpenHarmony 生态已积累超过 15,000 个应用、9 亿台设备（FSE'25 论文数据）。应用形态越丰富，页面状态、弹窗、异步加载与多设备适配带来的质量问题越难靠人工覆盖。Android 上的动态分析工具又无法直接迁移到 OpenHarmony：ArkTS/ArkUI 的声明式 UI、Ability 生命周期与 HDC/uitest 工具链都与 Android 不同。

现有测试方式各有边界：人工测试依赖业务经验，但重复操作成本高；脚本化测试稳定，但定位器、等待条件与页面变化需要持续维护；随机或规则探索能自动操作，却难以表达业务目标，也难以把一次探索沉淀为可复现资产。

单一控件树只能回答“有什么控件”，单一截图只能回答“看起来像什么”；把结构、视觉与运行上下文联合起来，才能判断页面处于什么业务状态、上一步动作是否真正生效。

三、方案

作品按三层组织：

1. 研究底座（**FSE'25**）：模块化动态分析框架，以控件树 + 截图 + 日志的多模态观察为基础，包含事件生成、设备执行、数据处理与可选代码插桩；以 PTG 记录“从哪个页面、通过什么操作到达哪里”；以统一策略管线支持随机、贪心 DFS、静态引导与 LLM 引导；执行结果沉淀为日志、截图、页面图与用例。

2. 系统级新增（**2026**）：HarmonyDevice 内部 API 与 haptest harmony CLI。API 以 observation 为中心，强制“动作前必须观察、动作后观察立即失效”，坐标动作绑定 observationId、前台 bundle、屏幕尺寸与 contentSig；CLI 以 JSON 输出、四类退出码与可执行的 nextStep 面向 shell/Python/Agent；serve 提供有状态 JSON-lines 会话。

3. 研究延伸：多模态 UI 遮挡诊断 HCO-Eye（ASE'25），以及 DevEco 性能管线逆向与掉帧测量设计。性能部分完成 hitrace → .htrace → trace_streamer → SQLite frame_slice 的链路分析与接口设计，报告只使用有数据来源的数字。

四、创新点

1. 多模态观察与系统级设备抽象：把控件树、截图、日志与任务语义放进同一次观察，再把 HDC/uitest/Hypium 的差异收敛到 HarmonyDevice 门面；强制 observation 生命周期、过期检测、歧义拒绝、跨进程租约、锁屏保护与认证字段保护，避免“猜着点”带来的假执行。多模态观察是入口：没有可复核的观察，状态理解、动作选择与报告都无从谈起。

2. **PTG** 驱动的智能测试闭环：观察 → 状态识别 → 候选动作 → 策略评分 → 受约束执行 → 更新 PTG，形成可回溯的页面迁移图；候选过滤、评分与执行解耦，新策略只需实现选择接口。

3. 面向 **Agent** 的接口形态：CLI 一次性命令与 serve 有状态会话并存，stdout 永远是 JSON，失败返回 errorCode/message/nextStep；同一套语义同时服务实验脚本、CI 与未来的 Agent 宿主。

4. 多模态遮挡诊断：HCO-Eye 联合截图视觉语义与控件树结构，在 HCOBench 152 个样本上取得 93.12% F1，并在商业应用中发现 34 个高开销遮挡案例（ASE'25）。

5. 性能观测的**"逐帧"**定义：不复用"fps 低于阈值"的代理指标，而是以 vsync 对齐的帧切片与 jankFlag 定义掉帧，保证报告中的每个数字都有数据来源。

五、完成度与验证

当前仓库已完成：多模态观察（控件树 + 截图 + 日志）、核心框架、PTG、策略管线、静态/LLM 引导、UI viewer、HarmonyDevice API、haptest harmony CLI、JSON-lines serve、真机 E2E 链路与 60 个单元测试。npm run build 通过，npm test -- --run 为 20 个测试文件、60 个测试全部通过；无设备时 E2E 以 SKIPPED 退出 0，不伪造设备结果。

研究验证（标注来源）：

来源	结果
FSE 2025 Industry Track《HapTest: The Dynamic Analysis Framework for OpenHarmony》	20 款热门商业应用、11 款发现问题，共 26 个此前未报告的运行问题：13 个 ArkTS 崩溃、10 个 C++ 崩溃、3 个卡死；实验设备 HUAWEI Mate60 / OpenHarmony 5.0.0
ASE 2025 Industry Track《An Empirical Study on UI Overlap in OpenHarmony Applications》	分析 100 款应用、33,262,624 个重叠实例；HCOBench 152 个样本、38 个正例；精确率 97.14%、召回率 89.47%、F1 93.12%；发现 34 个高开销遮挡案例
发明专利	《基于模块化架构的 OpenHarmony 应用动态分析框架及方法》，申请号 202510375959.1，申请人北京航空航天大学，2025-05-15 初步审查合格

六、工程规范与开源说明

仓库采用 TypeScript（CommonJS），src/ 编译到 lib/；单元测试使用 vitest，E2E 直接 require 构建产物；GitHub Actions 覆盖 Linux/Windows 构建。仓库保留 Apache-2.0 LICENSE 与上游版权头。HapTest 起源于 SMAT-Lab/HapTest 与华为开源 HapTest 工作，本作品在既有底座上完成 HarmonyOS 设备 API、CLI/会话协议、安全不变量、真机 E2E 与性能观测设计；论文与专利成果按上述来源标注，不将实验室集体成果表述为个人独立完成。

七、价值与展望

对开发者，HapTest 把一次探索变成可回放的用例与证据；对测试团队，它把重复操作、缺陷复现与回归维护收敛到同一套接口；对生态，它提供了一块可复用的 OpenHarmony 测试基础设施。下一步将完成帧解析与性能报告落地、扩展多设备与 CI 集成，并把设备 API 接入更多 Agent 宿主。